

IF2224 Teori Bahasa Formal dan Otomata

Milestone 2



13524121 - Billy Ontoseno Irawan

13524122 - Nathaniel Christian

13524139 - Azri Arzaq Pohan

13524142 - Rasyad Satyatma

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026

BAB 1

Dasar Teori

1.1 *Context Free Grammar (CFG)*

Context-Free Grammar (CFG) adalah suatu model formal yang mendeskripsikan struktur frasa suatu bahasa pemrograman. CFG didefinisikan secara formal sebagai

$$G = (V, T, P, S)$$

- V merupakan himpunan terbatas dari simbol non-terminal
- T merupakan himpunan terbatas dari simbol terminal
- P merupakan himpunan terbatas dari aturan produksi yang memiliki bentuk $A \rightarrow \alpha$ dengan A sebagai simbol non-terminal dan α sebagai string yang terdiri dari simbol terminal maupun non-terminal
- $S \in V$ merupakan simbol awal (*start symbol*) yang menjadi titik awal dari semua proses pembentukan string.

1.2 *Syntax Analysis (Parser)*

Syntax Analysis atau Parser bertugas untuk memeriksa urutan token yang dihasilkan oleh *lexer* agar sesuai dengan aturan tata bahasa dari bahasa pemrograman yang dikompilasi. Apabila ditemukan urutan token yang tidak sesuai dengan tata bahasa, parser akan mendeteksi dan menyatakan *syntax error*.

Parser akan membangun struktur sintaks seperti Parse Tree atau AST (*Abstract Syntax Tree*) yang merepresentasikan struktur hierarkis program sumber berdasarkan tata bahasa yang telah didefinisikan. Terdapat dua pendekatan dalam membuat Parse Tree:

1. Top Down

Parser memulai membangun pohon dari akar (simbol awal) lalu menurunkannya langkah demi langkah hingga cocok dengan token yang datang. Secara konseptual, pendekatan Top Down membangun *left derivation*. Terdapat dua macam pendekatan Top Down yaitu dengan backtracking dan tanpa backtracking. Dengan backtracking, algoritma yang biasa digunakan adalah algoritma brute force. Tanpa backtracking, algoritma yang biasa digunakan adalah algoritma Recursive Descent dan Predictive Parser (LL(1), LL(K)). Kelebihan pendekatan ini adalah transparansi. Struktur kode hampir 1:1 dengan tata bahasa. Selain itu, laporan kesalahan mudah disisipkan di posisi yang tepat dan mudah dirawat ketika tata bahasa berubah. Namun, kekuatan pengenalannya lebih terbatas. Jika tata bahasa mengandung ambiguitas atau pola yang secara alami lebih cocok

dipecahkan dari bawah (misalnya banyak produksi yang saling tumpang-tindih), pendekatan Top Down bisa memerlukan perubahan tata bahasa yang signifikan atau terjerumus ke backtracking yang tidak efisien.

2. Bottom Up

Parser membangun pohon dari daun ke atas menuju akar. Proses ini mencoba mereduksi input string kembali menjadi simbol awal. Parser mencari substring input yang persis cocok dengan sisi kanan sebuah produksi pada konteks yang tepat, lalu melakukan reduksi menjadi non-terminal pada sisi kirinya. Pendekatan ini biasanya menggunakan algoritma LR Parser (LR(0), SLR(1), LALR(1), CLR(1)). Keunggulan *Bottom Up* adalah daya ungkapnya luas. Resolusi ambiguitas operator dapat ditangani sistematis lewat deklarasi prioritas dan asosiativitas pada generator parser. Namun, kode hasil generator cenderung kurang intuitif bila dibandingkan *recursive descent*, dan ketika terjadi error sintaks jejaknya dapat lebih sulit dibaca karena keputusan parser tergantung keadaan automata dan tabel parse.

1.3 *Parse Tree*

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan tata bahasa yang digunakan. Struktur parse tree terdiri dari:

- Node akar (root), yaitu simbol awal tata bahasa.
- Node internal, yaitu simbol non-terminal.
- Node daun (leaf), yaitu simbol terminal

Setiap node internal beserta anak-anaknya merepresentasikan satu penerapan aturan produksi. Parse tree berguna untuk menunjukkan cara token-token hasil *lexical analysis* membentuk struktur program yang sesuai dengan tata bahasa. Jika sebuah string dapat menghasilkan lebih dari satu parse tree yang valid, maka tata bahasa tersebut bersifat ambigu

1.4 *Recursive Descent*

Recursive Descent adalah salah satu algoritma yang menggunakan pendekatan Top Down untuk membuat Parse Tree. Cara kerja:

1. Proses parsing dimulai dengan memanggil prosedur untuk simbol awal
2. Setiap prosedur bertanggung jawab untuk mengenali bagian dari input string yang cocok dengan non-terminal tersebut.
3. Ketika sebuah prosedur bertemu dengan non-terminal lain di aturan produksinya, ia akan memanggil prosedur yang bersesuaian.
4. Ketika bertemu simbol terminal, prosedur akan mencocokkannya dengan token input saat ini dan maju ke token berikutnya

Dengan strategi ini, aturan produksi yang digunakan harus bersifat tidak *left recursion* dan *non-deterministic*. Pendekatan ini mudah dibaca dan dirawat karena struktur kodenya paralel dengan struktur tata bahasa. *Recursive descent* biasanya dipakai dalam bentuk prediktif satu token agar berjalan mulus tanpa backtracking. Namun, tata bahasa harus disiapkan sehingga keputusan alternatif produksi dapat dibuat hanya dengan melihat token berikutnya.

Kelebihan utama *recursive descent* adalah transparansi dan modularitas. Karena setiap fungsi mencerminkan satu non-terminal, perubahan tata bahasa mudah diproyeksikan ke perubahan kode dan pengecekan pada titik-titik tertentu dapat disisipkan tanpa mengganggu keseluruhan alur.

BAB 2

Perancangan dan Implementasi

Untuk bahasa pemrograman Arion, parser diimplementasikan menggunakan bahasa C++ dengan mengikuti paradigma *Object-Oriented Programming*. Fokus utama yang menjadi bahasan pada bab ini mengenai algoritma *recursive descent*, tata bahasa yang digunakan, serta implementasi dari parser.

2.1 Bahasa Pemrograman

Implementasi dari parser yang digunakan pada program ini menggunakan bahasa pemrograman C++ dengan version C++11 dan dikompilasi menggunakan g++ melalui Makefile. Kami memilih bahasa ini karena C++ menggunakan paradigma *Object-Oriented Programming* secara native melalui class, struct, dan enum class. Program ini juga menggunakan komponen STL seperti `std::unordered_map` untuk melakukan pencarian berdasarkan keyword dengan kompleksitas $O(1)$ dan `std::string` untuk melakukan manipulasi string yang dapat membantu dalam implementasi dari tabel keyword dan juga logika *scanning*.

Struktur pada program ini dirancang secara modular untuk pembagian tanggung jawab yang jelas, seperti enum `TokenType` mendefinisikan seluruh jenis token yang dapat dikenali oleh program. Struct `Token` merepresentasikan entitas dari token. Kelas `Lexer` mengintegrasikan kedua komponen tersebut sekaligus meng-*handle* logika *scanning* dan juga transisi dari DFA.

2.2 Algoritma Recursive Descent

Algoritma recursive diimplementasikan pada kelas `Parser` menggunakan method-method yang sesuai dengan tata bahasa. Root dari Parse Tree yang akan dibentuk adalah suatu node bernama `<program>` yang dikonstruksi dengan method `parseProgram()` pada `main.cpp`

2.3 Class ParseNode

```
(inside) ./src/Parser/Parser.hpp
```

```
struct ParseNode {
    std::string name;
    std::string token;
    std::vector<ParseNode*> children;

    ParseNode(const std::string& n) : name(n) {}

    ~ParseNode() {
        for(auto c : children) delete c;
    }

    void add(ParseNode* c) {
```

```

        children.push_back(c);
    }

    void print(std::ostream& out, const std::string& prefix = "", bool isLast
= true) {
        out << prefix;
        if (prefix.empty()) {
            out << name;
        } else {
            out << (isLast ? "└─ " : "├─ ") << name;
        }
        if(!token.empty()) out << "(" << token << ")";
        out << std::endl;
        for (std::size_t i = 0; i < children.size(); i++) {
            children[i]->print(out, prefix + (isLast ? "    " : "│    "), i + 1
== children.size());
        }
    }
};

```

Implementasi di atas menunjukkan struktur data yang digunakan untuk merepresentasikan node dari hasil *syntax analysis*. Class ParseNode menyimpan 3 atribut:

1. name yang digunakan untuk mengenali jenis node
2. token yang berisi nilai dari token yang diproses
3. children yang merupakan vector child node dari node tersebut

2.4 Kelas Parser

Berikut implementasi dari kelas Parser:

(inside) ./src/Parser/Parser.hpp

```

class Parser {
private:
    Lexer& lexer;
    Token curr;
    Token advance();
    void expect(TokenType t, const std::string& msg="");

    ParseNode* programHeader();
    ParseNode* declarationPart();
    ParseNode* constDeclaration();
    ParseNode* constant();
    ParseNode* typeDeclaration();
    ParseNode* typeSpecification();
    ParseNode* arrayType();
    ParseNode* recordType();
    ParseNode* fieldList();
    ParseNode* fieldPart();
    ParseNode* subprogramDeclaration();
    ParseNode* procedureDeclaration();
    ParseNode* functionDeclaration();

```

```

    ParseNode* compoundStatement();
    ParseNode* statementList();
    ParseNode* statement();
    ParseNode* assignmentStatement();
    ParseNode* variable();
    ParseNode* componentVariable();
    ParseNode* indexList();
    ParseNode* procedureOrFunctionCall();
    ParseNode* parameterList();
    ParseNode* expression();
    ParseNode* simpleExpression();
    ParseNode* term();
    ParseNode* factor();
    ParseNode* rangeType();
    ParseNode* enumeratedType();
    ParseNode* block();
    ParseNode* formalParameterList();
    ParseNode* parameterGroup();
    ParseNode* ifStatement();
    ParseNode* caseStatement();
    ParseNode* caseBlock();
    ParseNode* whileStatement();
    ParseNode* repeatStatement();
    ParseNode* forStatement();
    ParseNode* relationalOperator();
    ParseNode* additiveOperator();
    ParseNode* multiplicativeOperator();

public:
    Parser(Lexer& l);
    ParseNode* parseProgram();
};

```

(inside) ./src/Parser/Parser.cpp

```

#include "Parser.hpp"
#include <stdexcept>
using namespace std;

Parser::Parser(Lexer& l) : lexer(l) {
    curr = lexer.getNextToken();
}

Token Parser::advance() {
    Token t = curr;
    curr = lexer.getNextToken();
    return t;
}

void Parser::expect(TokenType t, const string& msg) {
    if (curr.type != t) {
        string m = "Syntax error: unexpected token ";

```

```

        m += (lexer.tokenTypeToString(curr.type) + ", expected " +
lexer.tokenTypeToString(t));
        if(!msg.empty()) m += " (" + msg + ")";
        throw runtime_error(m);
    }
}

ParseNode* Parser::programHeader() {
    ParseNode* node = new ParseNode("<program-header>");

    if (curr.type == TokenType::PROGRAMSY) {
        node->add(new ParseNode("programsy"));
        advance();
        if (curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");
            id->token = curr.value;
            node->add(id);
            advance();
        } else {
            throw runtime_error("Syntax error: expected program name");
        }

        if(curr.type == TokenType::SEMICOLON) {
            node->add(new ParseNode("semicolon"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected semicolon after
program header");
        }
    }
    else {
        throw runtime_error("Syntax error: program must start with
'program'");
    }

    return node;
}

ParseNode* Parser::declarationPart() {
    ParseNode* node = new ParseNode("<declaration-part>");

    while(curr.type == TokenType::CONSTSY) {
        node->add(constDeclaration());
    }

    while(curr.type == TokenType::TYPESY) {
        node->add(typeDeclaration());
    }

    while(curr.type == TokenType::VARSY) {
        ParseNode* varnode = new ParseNode("<var-declaration>");
        varnode->add(new ParseNode("varsy"));
        advance();

        while (curr.type == TokenType::IDENT) {
            ParseNode* idlist = new ParseNode("<identifier-list>");

```



```

        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        idlist->add(id);
        advance();

        while(curr.type == TokenType::COMMA) {
            idlist->add(new ParseNode("comma"));
            advance();

            if(curr.type == TokenType::IDENT) {
                ParseNode* id2 = new ParseNode("ident");
                id2->token = curr.value;
                idlist->add(id2);
                advance();
            }
        }
        varnode->add(idlist);

        if (curr.type == TokenType::COLON) {
            varnode->add(new ParseNode("colon"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ':' in var
declaration");
        }

        varnode->add(typeSpecification());

        if (curr.type == TokenType::SEMICOLON) {
            varnode->add(new ParseNode("semicolon"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ';' in var
declaration");
        }
    }
    node->add(varnode);
}

while(curr.type == TokenType::PROCEDURESY || curr.type ==
TokenType::FUNCTIONSY) {
    node->add(subprogramDeclaration());
}

return node;
}

ParseNode* Parser::constDeclaration() {
    ParseNode* node = new ParseNode("<const-declaration>");

    if(curr.type == TokenType::CONSTSY) {
        node->add(new ParseNode("constsy"));
        advance();

        while(curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");

```

```

        id->token = curr.value;
        node->add(id); advance();

        if (curr.type == TokenType::EQL || curr.type==TokenType::EQL) {
            node->add(new ParseNode("eql"));
            advance();
        }
        node->add(constant());

        if (curr.type == TokenType::SEMICOLON) {
            node->add(new ParseNode("semicolon")); advance();
        }
        else break;
    }
}
return node;
}

ParseNode* Parser::constant() {
    ParseNode* node = new ParseNode("<constant>");

    if (curr.type == TokenType::INTCON || curr.type == TokenType::REALCON ||
curr.type == TokenType::CHARCON || curr.type == TokenType::STRING) {
        ParseNode* n = new ParseNode(lexer.tokenTypeToString(curr.type));
        n->token = curr.value;
        node->add(n); advance();
        return node;
    }

    if (curr.type == TokenType::PLUS || curr.type == TokenType::MINUS) {
        ParseNode* op = new ParseNode(lexer.tokenTypeToString(curr.type));
        advance();
        node->add(op);

        if (curr.type == TokenType::IDENT || curr.type == TokenType::INTCON ||
curr.type == TokenType::REALCON) {
            ParseNode* n = new ParseNode(lexer.tokenTypeToString(curr.type));
            n->token = curr.value;
            node->add(n); advance();
        }
        return node;
    }

    if (curr.type == TokenType::IDENT) {
        ParseNode* n=new ParseNode("ident");
        n->token=curr.value;
        node->add(n); advance();
        return node;
    }

    throw runtime_error("Syntax error: expected constant");
}

ParseNode* Parser::typeDeclaration() {
    ParseNode* node = new ParseNode("<type-declaration>");

```

```

    if (curr.type == TokenType::TYPESY) {
        node->add(new ParseNode("typesy")); advance();

        while (curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");
            id->token = curr.value;
            node->add(id); advance();

            if (curr.type == TokenType::EQL) {
                node->add(new ParseNode("eq1"));
                advance();
            }
            node->add(typeSpecification());

            if (curr.type == TokenType::SEMICOLON) {
                node->add(new ParseNode("semicolon"));
                advance();
            }
            else break;
        }
    }
    return node;
}

ParseNode* Parser::typeSpecification() {
    ParseNode* node = new ParseNode("<type>");

    if(curr.type == TokenType::ARRAYSY) {
        node->add(arrayType());
        return node;
    }
    if(curr.type == TokenType::LPARENT) {
        node->add(enumeratedType());
        return node;
    }
    if(curr.type == TokenType::RECORDSY) {
        node->add(recordType());
        return node;
    }

    if(curr.type == TokenType::PLUS || curr.type == TokenType::MINUS ||
        curr.type == TokenType::INTCON || curr.type == TokenType::REALCON ||
        curr.type == TokenType::CHARCON || curr.type == TokenType::STRING) {
        node->add(rangeType());
        return node;
    }

    if(curr.type == TokenType::IDENT) {
        ParseNode* const1 = constant();
        if (curr.type == TokenType::PERIOD) {
            ParseNode* rangeNode = new ParseNode("<range>");
            rangeNode->add(const1);

            rangeNode->add(new ParseNode("period"));
            advance();
            if (curr.type == TokenType::PERIOD) {

```

```

        rangeNode->add(new ParseNode("period"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected '...' in range");
    }
    rangeNode->add(constant());
    node->add(rangeNode);
} else {
    ParseNode* idNode = const1->children[0];
    const1->children.clear();
    delete const1;
    node->add(idNode);
}
return node;
}

return node;
}

ParseNode* Parser::arrayType() {
    ParseNode* node = new ParseNode("<array-type>");
    if(curr.type == TokenType::ARRAYSY) {
        node->add(new ParseNode("arraysy"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected 'array'");
    }
    if(curr.type == TokenType::LBRACK) {
        node->add(new ParseNode("lbrack"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected '[' in array
declaration");
    }
    if(curr.type == TokenType::INTCON || curr.type==TokenType::CHARCON ||
curr.type==TokenType::IDENT || curr.type == TokenType::PLUS || curr.type ==
TokenType::MINUS) {
        node->add(rangeType());
    } else {
        throw runtime_error("Syntax error: expected range in array
declaration");
    }
    if(curr.type == TokenType::RBRACK) {
        node->add(new ParseNode("rbrack"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected ']' after array range");
    }
    if(curr.type == TokenType::OFSY) {
        node->add(new ParseNode("ofsy"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected 'of' in array
declaration");
    }
}

```

```

        node->add(typeSpecification());
        return node;
    }

ParseNode* Parser::rangeType() {
    ParseNode* node = new ParseNode("<range>");
    node->add(constant());
    if (curr.type == TokenType::PERIOD) {
        node->add(new ParseNode("period"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected '.' in range");
    }
    if (curr.type == TokenType::PERIOD) {
        node->add(new ParseNode("period"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected '.' in range");
    }

    node->add(constant());
    return node;
}

ParseNode* Parser::enumeratedType() {
    ParseNode* node = new ParseNode("<enumerated>");

    if(curr.type == TokenType::LPARENT) {
        node->add(new ParseNode("lparent")); advance();

        if (curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");
            id->token = curr.value;
            node->add(id); advance();
        } else {
            throw runtime_error("Syntax error: expected identifier in
enumerated type");
        }

        while (curr.type == TokenType::COMMA) {
            node->add(new ParseNode("comma")); advance();

            if(curr.type == TokenType::IDENT) {
                ParseNode* id = new ParseNode("ident");
                id->token = curr.value;
                node->add(id); advance();
            } else {
                throw runtime_error("Syntax error: expected identifier after
comma in enumerated type");
            }
        }
        if(curr.type == TokenType::RPARENT) {
            node->add(new ParseNode("rparent"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ')' in enumerated

```

```

type");
    }
    }
    return node;
}

ParseNode* Parser::recordType() {
    ParseNode* node = new ParseNode("<record-type>");

    if (curr.type == TokenType::RECORDSY) {
        node->add(new ParseNode("recordsy")); advance();
    }
    node->add(fieldList());
    if (curr.type == TokenType::ENDSY) {
        node->add(new ParseNode("endsy")); advance();
    } else {
        throw runtime_error("Syntax error: expected 'end' after record
fields");
    }

    return node;
}

ParseNode* Parser::fieldList() {
    ParseNode* node = new ParseNode("<field-list>");
    node->add(fieldPart());

    while (curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon")); advance();
        node->add(fieldPart());
    }
    return node;
}

ParseNode* Parser::fieldPart() {
    ParseNode* node = new ParseNode("<field-part>");
    ParseNode* idlist = new ParseNode("<identifier-list>");

    if (curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        idlist->add(id); advance();
    } else {
        throw runtime_error("Syntax error: expected identifier in record
field");
    }

    while (curr.type == TokenType::COMMA) {
        idlist->add(new ParseNode("comma")); advance();

        if (curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");
            id->token = curr.value;
            idlist->add(id); advance();
        } else {

```

```

        throw runtime_error("Syntax error: expected identifier after comma
in record field");
    }
}
node->add(idlist);

if (curr.type == TokenType::COLON) {
    node->add(new ParseNode("colon"));
    advance();
} else {
    throw runtime_error("Syntax error: expected ':' in record field");
}
node->add(typeSpecification());
return node;
}

ParseNode* Parser::subprogramDeclaration() {
    ParseNode* node = new ParseNode("<subprogram-declaration>");

    if(curr.type == TokenType::PROCEDURESY) {
        node->add(procedureDeclaration());
    } else if(curr.type == TokenType::FUNCTIONSY) {
        node->add(functionDeclaration());
    }
    return node;
}

ParseNode* Parser::procedureDeclaration() {
    ParseNode* node = new ParseNode("<procedure-declaration>");

    if(curr.type == TokenType::PROCEDURESY) {
        node->add(new ParseNode("proceduresy"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected 'procedure'");
    }

    if(curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        node->add(id);
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected procedure name");
    }

    if(curr.type == TokenType::LPARENT) {
        node->add(formalParameterList());
    }

    if(curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
    }
}

```

```

    }
    else {
        throw runtime_error("Syntax error: expected semicolon after procedure
header");
    }

    node->add(block());

    if(curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected semicolon after procedure
block");
    }

    return node;
}

ParseNode* Parser::functionDeclaration() {
    ParseNode* node = new ParseNode("<function-declaration>");

    if(curr.type == TokenType::FUNCTIONSY) {
        node->add(new ParseNode("functionsy"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected 'function'");
    }

    if(curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        node->add(id);
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected function name");
    }

    if(curr.type == TokenType::LPARENT) {
        node->add(formalParameterList());
    }

    if(curr.type == TokenType::COLON) {
        node->add(new ParseNode("colon"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected colon before function
return type");
    }

    if (curr.type == TokenType::IDENT) {
        ParseNode* returnType = new ParseNode("ident");

```



```

        returnType->token = curr.value;
        node->add(returnType);
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected function return type");
    }

    if(curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected semicolon after function
header");
    }

    node->add(block());

    if(curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
    }
    else {
        throw runtime_error("Syntax error: expected semicolon after function
block");
    }

    return node;
}

ParseNode* Parser::block() {
    ParseNode* node = new ParseNode("<block>");

    node->add(declarationPart());
    node->add(compoundStatement());
    return node;
}

ParseNode* Parser::formalParameterList() {
    ParseNode* node = new ParseNode("<formal-parameter-list>");
    if(curr.type == TokenType::LPARENT) {
        node->add(new ParseNode("lparent"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected '(' in formal parameter
list");
    }
    node->add(parameterGroup());
    while(curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
        node->add(parameterGroup());
    }
    if(curr.type == TokenType::RPARENT) {
        node->add(new ParseNode("rparent"));
    }
}

```

```

        advance();
    } else {
        throw runtime_error("Syntax error: expected ')' in formal parameter
list");
    }
    return node;
}

ParseNode* Parser::parameterGroup() {
    ParseNode* node = new ParseNode("<parameter-group>");
    ParseNode* idlist = new ParseNode("<identifier-list>");
    if(curr.type == TokenType::IDENT) {
        ParseNode* id=new ParseNode("ident");
        id->token=curr.value;
        idlist->add(id); advance();
    } else {
        throw runtime_error("Syntax error: expected identifier in parameter
group");
    }
    while(curr.type == TokenType::COMMA) {
        idlist->add(new ParseNode("comma"));
        advance();
        if(curr.type==TokenType::IDENT){
            ParseNode* id=new ParseNode("ident");
            id->token=curr.value;
            idlist->add(id); advance();
        } else {
            throw runtime_error("Syntax error: expected identifier after comma
in parameter group");
        }
    }
    node->add(idlist);
    if(curr.type == TokenType::COLON) {
        node->add(new ParseNode("colon"));
        advance();
    } else {
        throw runtime_error("Syntax error: expected ':' in parameter group");
    }
    if(curr.type == TokenType::IDENT || curr.type==TokenType::ARRAYSY) {
        node->add(typeSpecification());
    } else {
        throw runtime_error("Syntax error: expected type specification in
parameter group");
    }
    return node;
}

ParseNode* Parser::compoundStatement() {
    ParseNode* node = new ParseNode("<compound-statement>");

    if(curr.type == TokenType::BEGINSY) {
        node->add(new ParseNode("beginsy"));
        advance();
        node->add(statementList());
        if(curr.type == TokenType::ENDSY) {
            node->add(new ParseNode("endsy"));

```

```

        advance();
    } else {
        throw runtime_error("Syntax error: expected 'end'");
    }
} else {
    throw runtime_error("Syntax error: expected 'begin'");
}
return node;
}

ParseNode* Parser::statementList() {
    ParseNode* node = new ParseNode("<statement-list>");

    while (curr.type != TokenType::ENDSY && curr.type !=
TokenType::END_OF_FILE) {
        node->add(statement());
        if(curr.type == TokenType::SEMICOLON) {
            node->add(new ParseNode("semicolon"));
            advance();
        } else {
            break;
        }
    }
    return node;
}

ParseNode* Parser::statement() {
    if(curr.type == TokenType::BEGINSY) return compoundStatement();
    if(curr.type == TokenType::IFSY) return ifStatement();
    if(curr.type == TokenType::CASESY) return caseStatement();
    if(curr.type == TokenType::WHILESY) return whileStatement();
    if(curr.type == TokenType::REPEATSY) return repeatStatement();
    if(curr.type == TokenType::FORSY) return forStatement();

    if(curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        advance();

        if (curr.type == TokenType::LPARENT) {
            ParseNode* call = new ParseNode("<procedure/function-call>");
            call->add(id);
            call->add(new ParseNode("lparent"));
            advance();
            if(curr.type != TokenType::RPARENT) {
                call->add(parameterList());
            }
            if(curr.type == TokenType::RPARENT) {
                call->add(new ParseNode("rparent"));
                advance();
            } else {
                throw runtime_error("Syntax error: expected ')' after
procedure/function call");
            }
            return call;
        } else {

```

```

        ParseNode* varToUse = id;
        if (curr.type == TokenType::LBRACK || curr.type ==
TokenType::PERIOD) {
            ParseNode* var = new ParseNode("<variable>");
            var->add(id);
            while(curr.type == TokenType::LBRACK || curr.type ==
TokenType::PERIOD) {
                var->add(componentVariable());
            }
            varToUse = var;
        }

        if(curr.type == TokenType::BECOMES) {
            ParseNode* node = new ParseNode("<assignment-statement>");
            node->add(varToUse);
            node->add(new ParseNode("becomes"));
            advance();
            node->add(expression());
            return node;
        } else if (varToUse == id) {
            ParseNode* call = new ParseNode("<procedure/function-call>");
            call->add(id);
            return call;
        }

        throw runtime_error("Syntax error: expected ':= ' in assignment");
    }
}

ParseNode* node = new ParseNode("<statement>");
ParseNode* t = new ParseNode(lexer.tokenTypeToString(curr.type));
t->token = curr.value;
node->add(t); advance();
return node;
}

ParseNode* Parser::variable() {
    if(curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        advance();

        // check for components (array index or record field)
        // ident + (component-variable)*
        if(curr.type == TokenType::LBRACK || curr.type == TokenType::PERIOD) {
            ParseNode* var= new ParseNode("<variable>");
            var->add(id);

            while(curr.type == TokenType::LBRACK || curr.type ==
TokenType::PERIOD) {
                var->add(componentVariable());
            }

            return var;
        }
        return id;
    }
}

```

```

    }
    throw runtime_error("Syntax error: expected variable (identifier)");
}

ParseNode* Parser::componentVariable() {
    ParseNode* node = new ParseNode("<component-variable>");

    if (curr.type == TokenType::LBRACK) {
        // array index: [index-list]
        node->add(new ParseNode("lbrack"));
        advance();
        node->add(indexList());

        if (curr.type == TokenType::RBRACK) {
            node->add(new ParseNode("rbrack"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ']' after index
list");
        }
    } else if (curr.type == TokenType::PERIOD) {
        // .ident
        node->add(new ParseNode("period"));
        advance();
        if (curr.type == TokenType::IDENT) {
            ParseNode* id = new ParseNode("ident");
            id->token = curr.value;
            node->add(id);
            advance();
        } else {
            throw runtime_error("Syntax error: expected identifier after
'.'");
        }
    }
    return node;
}

ParseNode* Parser::indexList() {
    ParseNode* node = new ParseNode("<index-list>");

    node->add(factor());
    while (curr.type == TokenType::COMMA) {
        node->add(new ParseNode("comma"));
        advance();
        node->add(factor());
    }
    return node;
}

ParseNode* Parser::assignmentStatement() {
    ParseNode* node = new ParseNode("<assignment-statement>");
    node->add(variable());
    if (curr.type == TokenType::BECOMES) {
        node->add(new ParseNode("becomes"));
        advance();
    } else {

```

```

        throw runtime_error("Syntax error: expected ':' in assignment");
    }
    node->add(expression());
    return node;
}

ParseNode* Parser::ifStatement() {
    ParseNode* node = new ParseNode("<if-statement>");

    if (curr.type == TokenType::IFSY) {
        node->add(new ParseNode("ifsy"));
        advance();
    }
    node->add(expression());
    if (curr.type == TokenType::THENSY) {
        node->add(new ParseNode("thensy"));
        advance();
    }
    node->add(statement());
    if (curr.type == TokenType::ELSESY) {
        node->add(new ParseNode("elsesy"));
        advance();
        node->add(statement());
    }
    return node;
}

ParseNode* Parser::caseBlock() {
    ParseNode* node = new ParseNode("<case-block>");

    node->add(constant());
    while (curr.type == TokenType::COMMA) {
        node->add(new ParseNode("comma"));
        advance();
        node->add(constant());
    }
    if (curr.type == TokenType::COLON) {
        node->add(new ParseNode("colon"));
        advance();
    }
    node->add(statement());
    if (curr.type == TokenType::SEMICOLON) {
        node->add(new ParseNode("semicolon"));
        advance();
    }
    return node;
}

ParseNode* Parser::caseStatement() {
    ParseNode* node = new ParseNode("<case-statement>");

    if (curr.type == TokenType::CASESY) {
        node->add(new ParseNode("casesy"));
        advance();
    }
    node->add(expression());
}

```

```

        if(curr.type == TokenType::OFSY) {
            node->add(new ParseNode("ofsy"));
            advance();
        }
        while(curr.type != TokenType::ENDSY && curr.type !=
TokenType::END_OF_FILE) {
            node->add(caseBlock());
        }
        if(curr.type == TokenType::ENDSY) {
            node->add(new ParseNode("endsy"));
            advance();
        }
        return node;
    }
}

ParseNode* Parser::whileStatement() {
    ParseNode* node = new ParseNode("<while-statement>");
    if(curr.type == TokenType::WHILESY) {
        node->add(new ParseNode("whilesy"));
        advance();
    }
    node->add(expression());
    if(curr.type == TokenType::DOSY) {
        node->add(new ParseNode("dosy"));
        advance();
    }
    node->add(statement());
    return node;
}

ParseNode* Parser::repeatStatement() {
    ParseNode* node = new ParseNode("<repeat-statement>");
    if(curr.type == TokenType::REPEATSY) {
        node->add(new ParseNode("repeatsy"));
        advance();
    }
    node->add(statementList());
    if(curr.type == TokenType::UNTILSY) {
        node->add(new ParseNode("untilsy"));
        advance();
    }
    node->add(expression());
    return node;
}

ParseNode* Parser::forStatement() {
    ParseNode* node = new ParseNode("<for-statement>");

    if(curr.type == TokenType::FORSY) {
        node->add(new ParseNode("forsy"));
        advance();
    }
    if(curr.type == TokenType::IDENT) {
        ParseNode* id=new ParseNode("ident");
        id->token=curr.value;
        node->add(id);
    }
}

```

```

        advance();
    }
    if(curr.type == TokenType::BECOMES) {
        node->add(new ParseNode("becomes"));
        advance();
    }
    node->add(expression());
    if(curr.type == TokenType::TOSY) {
        node->add(new ParseNode("tosy"));
        advance();
    } else if(curr.type == TokenType::DOWNTOSY) {
        node->add(new ParseNode("downtosy"));
        advance();
    }
    node->add(expression());
    if(curr.type == TokenType::DOSY) {
        node->add(new ParseNode("dosy"));
        advance();
    }
    node->add(statement());
    return node;
}

ParseNode* Parser::procedureOrFunctionCall() {
    ParseNode* node = new ParseNode("<procedure/function-call>");
    if(curr.type == TokenType::IDENT) {
        ParseNode* id=new ParseNode("ident");
        id->token=curr.value;
        node->add(id);
        advance();
    } else {
        throw runtime_error("Syntax error: expected procedure/function name");
    }
    if(curr.type == TokenType::LPARENT) {
        node->add(new ParseNode("lparent"));
        advance();
        if(curr.type != TokenType::RPARENT) {
            node->add(parameterList());
        }
        if(curr.type == TokenType::RPARENT) {
            node->add(new ParseNode("rparent"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ')'");
        }
    }
    return node;
}

ParseNode* Parser::parameterList() {
    ParseNode* node = new ParseNode("<parameter-list>");
    node->add(expression());
    while(curr.type == TokenType::COMMA) {
        node->add(new ParseNode("comma"));
        advance();
        node->add(expression());
    }
}

```



```

    }
    return node;
}

ParseNode* Parser::expression() {
    ParseNode* node = new ParseNode("<expression>");
    node->add(simpleExpression());

    if(curr.type == TokenType::EQL || curr.type == TokenType::NEQ ||
        curr.type == TokenType::GTR || curr.type == TokenType::GEQ ||
        curr.type == TokenType::LSS || curr.type == TokenType::LEQ) {
        node->add(relationalOperator());
        node->add(simpleExpression());
    }
    return node;
}

ParseNode* Parser::simpleExpression() {
    ParseNode* node = new ParseNode("<simple-expression>");

    if(curr.type == TokenType::PLUS || curr.type == TokenType::MINUS) {
        ParseNode* op = new ParseNode(lexer.tokenTypeToString(curr.type));
        op->token = curr.value;
        advance();
        node->add(op);
    }

    node->add(term());

    while(curr.type == TokenType::PLUS || curr.type == TokenType::MINUS ||
        curr.type == TokenType::ORSY) {
        node->add(additiveOperator());
        node->add(term());
    }
    return node;
}

ParseNode* Parser::term() {
    ParseNode* node = new ParseNode("<term>");

    node->add(factor());
    while(curr.type == TokenType::TIMES || curr.type == TokenType::RDIV ||
        curr.type == TokenType::IDIV || curr.type == TokenType::IMOD ||
        curr.type == TokenType::ANDSY) {
        node->add(multiplicativeOperator());
        node->add(factor());
    }
    return node;
}

ParseNode* Parser::factor() {
    ParseNode* node = new ParseNode("<factor>");
    if(curr.type == TokenType::NOTSY) {
        node->add(new ParseNode("notsy"));
        advance();
    }
}

```

```

        node->add(factor());
        return node;
    }
    if(curr.type == TokenType::INTCON || curr.type == TokenType::REALCON ||
        curr.type == TokenType::STRING || curr.type == TokenType::CHARCON) {
        ParseNode* n = new ParseNode(lexer.tokenTypeToString(curr.type));
        n->token = curr.value;
        node->add(n);
        advance();
        return node;
    }
    if(curr.type == TokenType::LPARENT) {
        node->add(new ParseNode("lparent"));
        advance();
        node->add(expression());
        if(curr.type == TokenType::RPARENT) {
            node->add(new ParseNode("rparent"));
            advance();
        } else {
            throw runtime_error("Syntax error: expected ");
        }
        return node;
    }
    if(curr.type == TokenType::IDENT) {
        ParseNode* id = new ParseNode("ident");
        id->token = curr.value;
        advance();

        if (curr.type == TokenType::LPARENT) {
            ParseNode* call = new ParseNode("<procedure/function-call>");
            call->add(id);
            call->add(new ParseNode("lparent"));
            advance();
            if(curr.type != TokenType::RPARENT) {
                call->add(parameterList());
            }
            if(curr.type == TokenType::RPARENT) {
                call->add(new ParseNode("rparent"));
                advance();
            } else {
                throw runtime_error("Syntax error: expected ')' after function
call");
            }
            node->add(call);
            return node;
        } else {
            if (curr.type == TokenType::LBRACK || curr.type ==
TokenType::PERIOD) {
                ParseNode* var = new ParseNode("<variable>");
                var->add(id);
                while(curr.type == TokenType::LBRACK || curr.type ==
TokenType::PERIOD) {
                    var->add(componentVariable());
                }
                node->add(var);
            } else {

```

```

        node->add(id);
    }

    return node;
}

// unknown factor
ParseNode* n = new ParseNode("unknown");
n->token = curr.value;
node->add(n);
advance();
return node;
}

ParseNode* Parser::relationalOperator() {
    ParseNode* node = new ParseNode("<relational-operator>");

    if(curr.type==TokenType::EQL||curr.type==TokenType::NEQ||curr.type==TokenType::GTR||curr.type==TokenType::GEQ||curr.type==TokenType::LSS||curr.type==TokenType::LEQ) {
        node->add(new ParseNode(lexer.tokenTypeToString(curr.type)));
        advance();
    }
    return node;
}

ParseNode* Parser::additiveOperator() {
    ParseNode* node = new ParseNode("<additive-operator>");

    if(curr.type==TokenType::PLUS||curr.type==TokenType::MINUS||curr.type==TokenType::ORSY) {
        node->add(new ParseNode(lexer.tokenTypeToString(curr.type)));
        advance();
    }
    return node;
}

ParseNode* Parser::multiplicativeOperator() {
    ParseNode* node = new ParseNode("<multiplicative-operator>");

    if(curr.type==TokenType::TIMES||curr.type==TokenType::RDIV||curr.type==TokenType::IDIV||curr.type==TokenType::IMOD||curr.type==TokenType::ANDSY) {
        node->add(new ParseNode(lexer.tokenTypeToString(curr.type)));
        advance();
    }
    return node;
}

ParseNode* Parser::parseProgram() {
    ParseNode* root = new ParseNode("<program>");
    root->add(programHeader());
    root->add(declarationPart());
    root->add(compoundStatement());
    if(curr.type == TokenType::PERIOD) {
        root->add(new ParseNode("period"));
    }
}

```

```

    }
    return root;
}

```

Method	Aturan Produksi
parseProgram()	program → program-header + declaration-part + compound-statement + period
programHeader()	programsy + ident + semicolon
declarationPart()	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*
constDeclaration()	constsy + (ident + eql + constant + semicolon)+
constant()	charcon string [(plus minus)? + (ident intcon realcon)]
typeDeclaration()	typesy + (ident + eql + type + semicolon)+
declarationPart()	varsy + (identifier-list + colon + type + semicolon)+
declarationPart()	ident (comma + ident)*
typeSpecification()	ident array-type range enumerated record-type
arrayType()	arraysy + lbrack + (range ident) + rbrack + ofsy + type
rangeType()	constant + period + period + constant
enumeratedType()	lparent + ident + (comma + ident)* + rparent
recordType()	recordsy + field-list + endsy

fieldList()	field-part + (semicolon + field-part)*
fieldPart()	identifier-list + colon + type
subprogramDeclaration()	procedure-declaration function-declaration
procedureDeclaration()	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon
functionDeclaration()	functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon+ block + semicolon
block()	declaration-part + compound-statement
formalParameterList()	lparent + parameter-group + (semicolon + parameter-group)* + rparent
parameterGroup()	identifier-list + colon + (ident array-type)
compoundStatement()	beginsy + statement-list + endsy
statementList()	statement (semicolon + statement)*
statement()	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?
variable()	ident + (component-variable)*
componentVariable()	(lbrack + index-list + rbrack) (period + ident)
indexList()	(intcon charcon ident) + (comma + index-list)*
assignmentStatement()	variable + becomes + expression
ifStatement()	ifsy + expression + thensy + statement + (elsy + statement)?

caseStatement()	casesy + expression + ofsy + case-block + endsy
caseBlock()	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*
whileStatement()	whilesy + expression + dosy + statement
repeatStatement()	repeatsy + statement-list + untilsy + expression
forStatement()	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement
procedureOrFunctionCall()	ident + (lparent + parameter-list? + rparent)?
parameterList()	expression (comma + expression)*
expression()	simple-expression (relational-operator + simple-expression)?
simpleExpression()	(plus minus)? term (additive-operator + term)*
term()	factor (multiplicative-operator + factor)*
factor()	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable
relationalOperator()	eq neq gtr geq lss leq
additiveOperator()	plus minus orsy
multiplicativeOperator()	times rdiv idiv imod andsy

Masing-masing implementasi method dari kelas Parser adalah implementasi *production rule* yang ada pada tabel diatas. Beberapa node (dan sub-node) mungkin tergabung dalam satu method, contohnya var-declaration dan identifier-list yang masuk ke implementasi method declarationPart().

2.5 Alur Program

Berikut adalah kode main.cpp dari program:

```
./src/main.cpp

#include "Lexer/Lexer.hpp"
#include "Parser/Parser.hpp"
#include <string>
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char* argv[]) {
    if(argc < 2) {
        cerr << "Usage: ./bin/arion-compiler [--parse] <input-file.txt>" <<
endl;

        cerr << "  Default: lexer only" << endl;
        cerr << "  --parse: run lexer + parser and output parse tree" << endl;
        cerr << "  Example: ./bin/arion-compiler test.txt" << endl;
        cerr << "  Example: ./bin/arion-compiler --parse test.txt" << endl;
        return 1;
    }

    bool parseMode = false;
    string filename;

    if(argc >= 3 && string(argv[1]) == "--parse") {
        parseMode = true;
        filename = argv[2];
    } else {
        filename = argv[1];
    }

    try {
        if(parseMode) {
            // Parser mode
            Lexer lexer(filename);
            Parser parser(lexer);
            ParseNode* tree = parser.parseProgram();

            // Print tree to stdout
            tree->print(cout);

            // Save tree to output file
            string outfile = filename;
            size_t lastDot = outfile.rfind('.');
            if(lastDot != string::npos) {
                outfile = outfile.substr(0, lastDot) + "_parse.txt";
            } else {
                outfile = outfile + "_parse.txt";
            }
            ofstream out(outfile);
            tree->print(out);
        }
    }
```

```

        out.close();
        cerr << "Parse tree saved to: " << outfile << endl;

        delete tree;
    } else {
        // Lexer mode (default)
        Lexer lexer(filename);
        Token token = lexer.getNextToken();
        while(token.type != TokenType::END_OF_FILE) {
            string typeName = lexer.tokenTypeToString(token.type);

            if(token.type == TokenType::IDENT || token.type ==
TokenType::INTCON || token.type == TokenType::REALCON) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else if (token.type == TokenType::STRING || token.type ==
TokenType::CHARCON) {
                cout << typeName << " ('" << token.value << "')" << endl;
            }
            else if (token.type == TokenType::UNKNOWN) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else {
                cout << typeName << endl;
            }
            token = lexer.getNextToken();
        }
    }
}
catch (const exception& e) {
    cerr << "Error: " << e.what() << endl;
    return 1;
}
return 0;
}

```

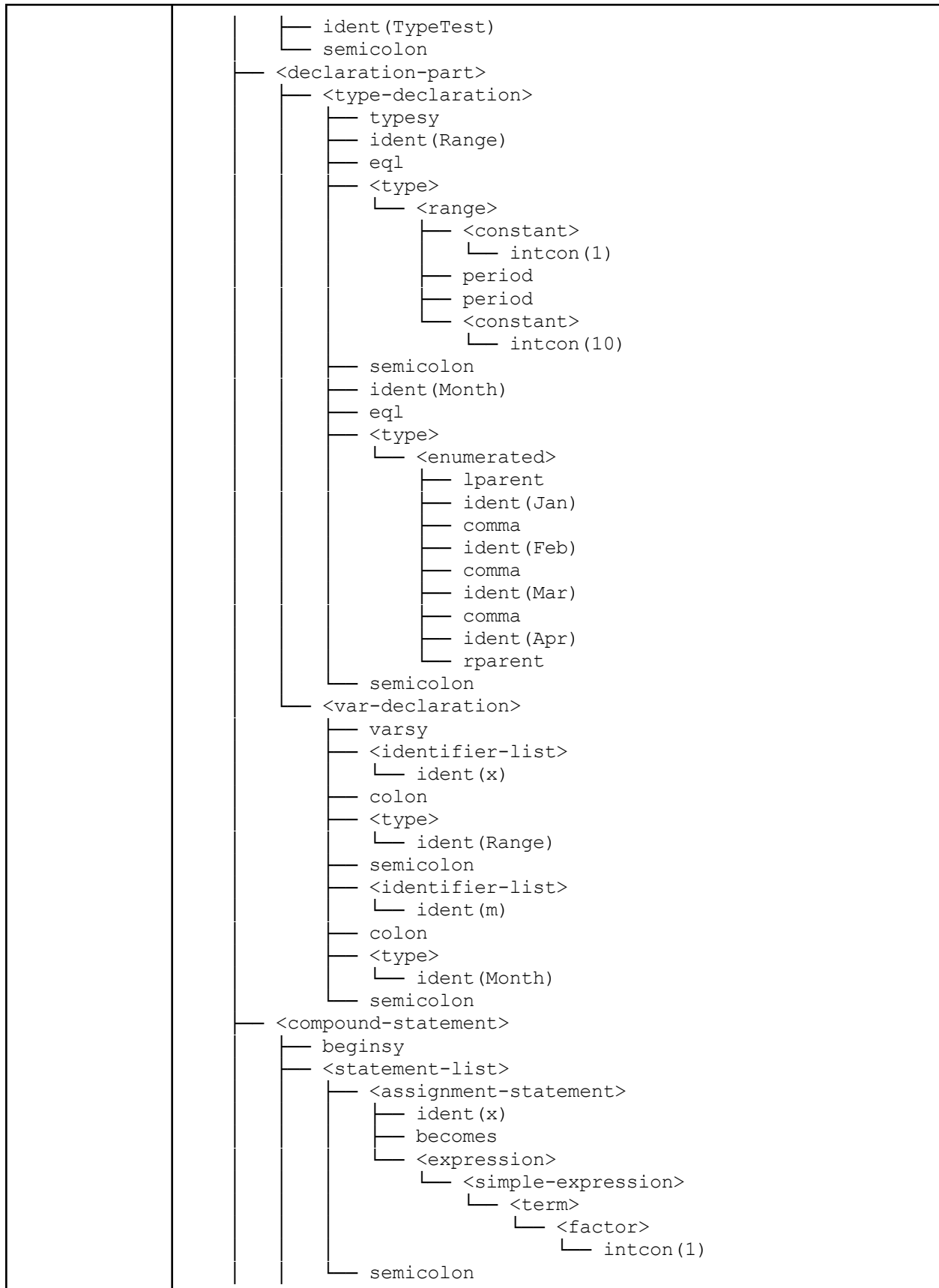
Program dimulai dari fungsi main() yang membaca argumen baris perintah dan memilih mode kerja: mode default hanya menjalankan Lexer untuk melakukan tokenisasi, sedangkan jika opsi --parse diberikan program juga membangun Parser untuk menghasilkan pohon parse. Pada mode lexer, program membuat objek Lexer dengan berkas input lalu memanggil getNextToken() berulang kali sampai akhir berkas, mencetak tipe token dan nilai yang relevan untuk kebutuhan debugging. Pada mode parser, program membuat Lexer dan Parser, memanggil parseProgram() untuk mendapatkan struktur pohon, lalu mencetak pohon tersebut ke layar dan menyimpannya ke berkas output yang dinamai dari input dengan suffix _parse.txt. Seluruh operasi dibungkus dalam blok try/catch sehingga kesalahan leksikal atau sintaksis ditangani dengan mencetak pesan error dan keluar dengan kode gagal.

BAB 3

Pengujian

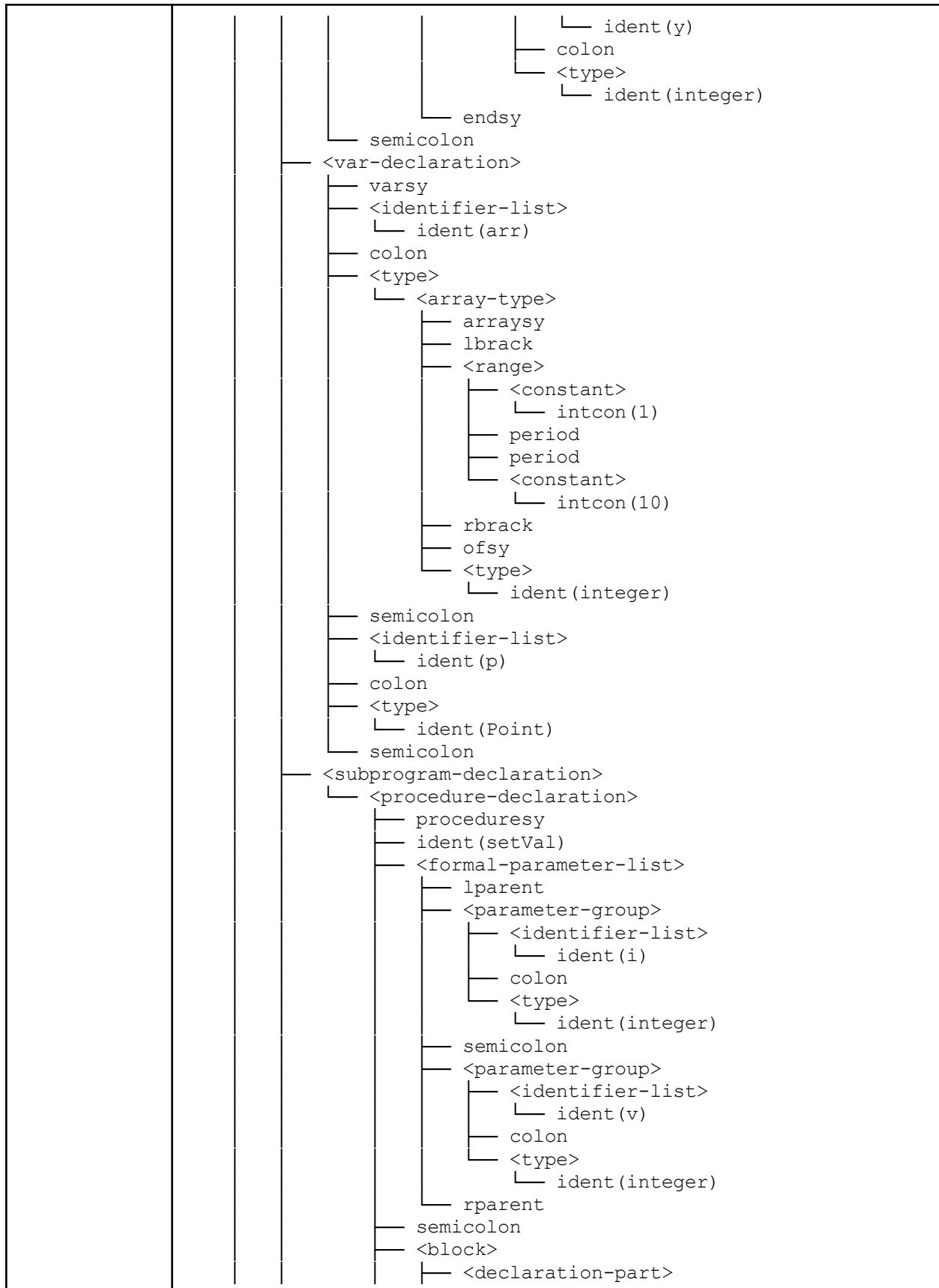
input-1.txt	<pre> <program> ├── <program-header> │ ├── programsy │ ├── ident (Hello) │ └── semicolon ├── <declaration-part> │ └── <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ ├── ident (a) │ │ ├── comma │ │ └── ident (b) │ ├── colon │ ├── <type> │ │ └── ident (integer) │ └── semicolon └── <compound-statement> ├── beginsy ├── <statement-list> │ ├── <assignment-statement> │ │ ├── ident (a) │ │ ├── becomes │ │ └── <expression> │ │ └── <simple-expression> │ │ └── <term> │ │ └── <factor> │ │ └── intcon (5) │ ├── semicolon │ ├── <assignment-statement> │ │ ├── ident (b) │ │ ├── becomes │ │ └── <expression> │ │ └── <simple-expression> │ │ ├── <term> │ │ │ ├── <factor> │ │ │ └── ident (a) │ │ ├── <additive-operator> │ │ │ └── plus │ │ └── <term> │ │ └── <factor> │ │ └── intcon (10) │ └── semicolon └── <procedure/function-call> ├── ident (writeln) ├── lparent ├── <parameter-list> │ └── <expression> │ └── <simple-expression> │ └── <term> │ └── <factor> </pre>
-------------	---

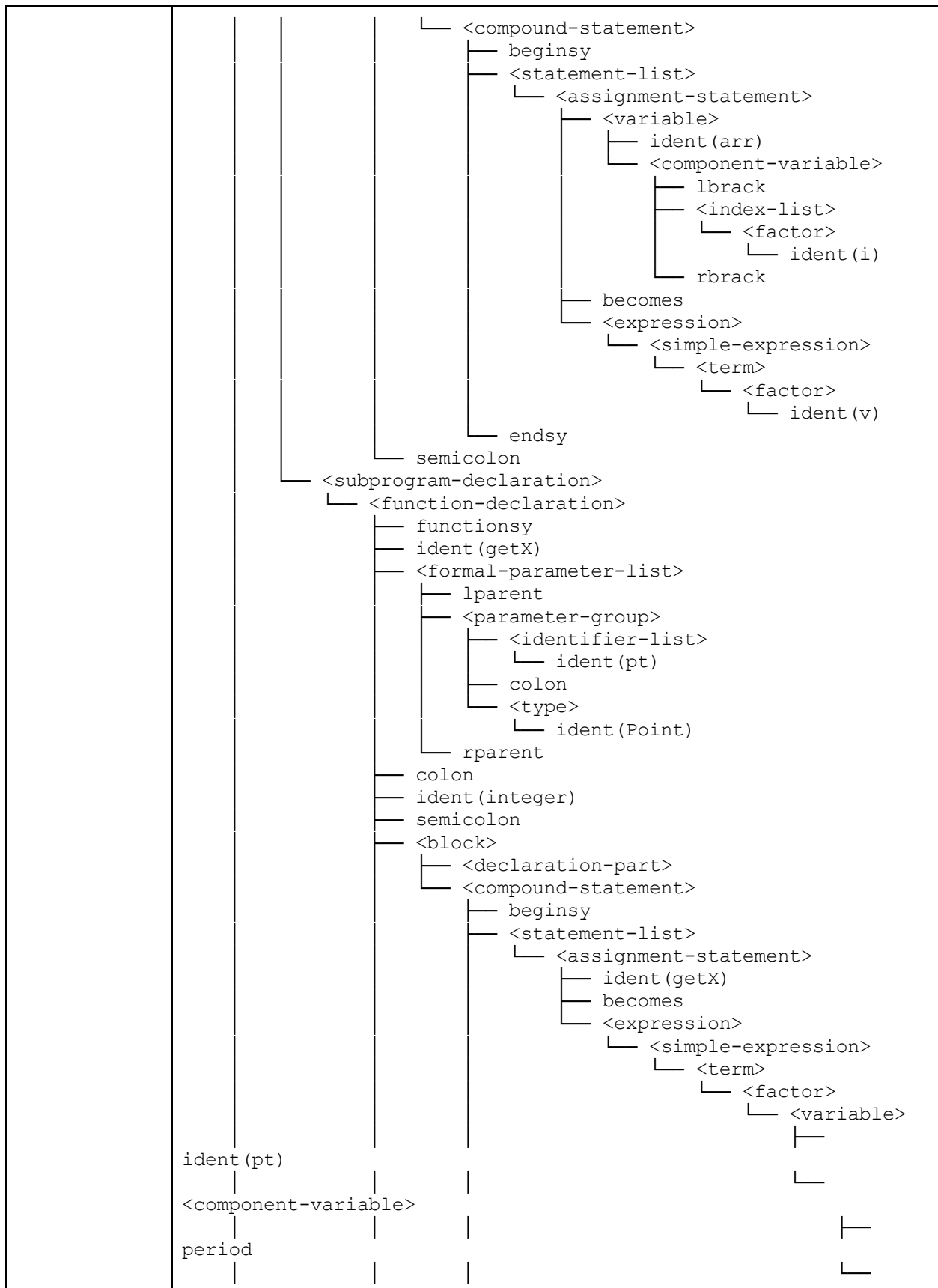
	<pre> └── string(Result =) ├── comma └── <expression> └── <simple-expression> └── <term> └── <factor> └── ident(b) └── rparent └── semicolon └── endsy └── period </pre>
input-2.txt	<pre> <program> ├── <program-header> │ ├── programsy │ ├── ident(ConstTest) │ └── semicolon ├── <declaration-part> │ └── <const-declaration> │ ├── constsy │ ├── ident(A) │ ├── eql │ ├── <constant> │ │ └── intcon(100) │ ├── semicolon │ ├── ident(B) │ ├── eql │ ├── <constant> │ │ ├── plus │ │ └── realcon(10.5) │ ├── semicolon │ ├── ident(C) │ ├── eql │ ├── <constant> │ │ ├── minus │ │ └── intcon(20) │ ├── semicolon │ ├── ident(D) │ ├── eql │ ├── <constant> │ │ └── charcon(x) │ ├── semicolon │ ├── ident(E) │ ├── eql │ ├── <constant> │ │ └── string(hello) │ └── semicolon ├── <compound-statement> │ ├── beginsy │ ├── <statement-list> │ └── endsy └── period </pre>
input-3.txt	<pre> <program> ├── <program-header> └── programsy </pre>

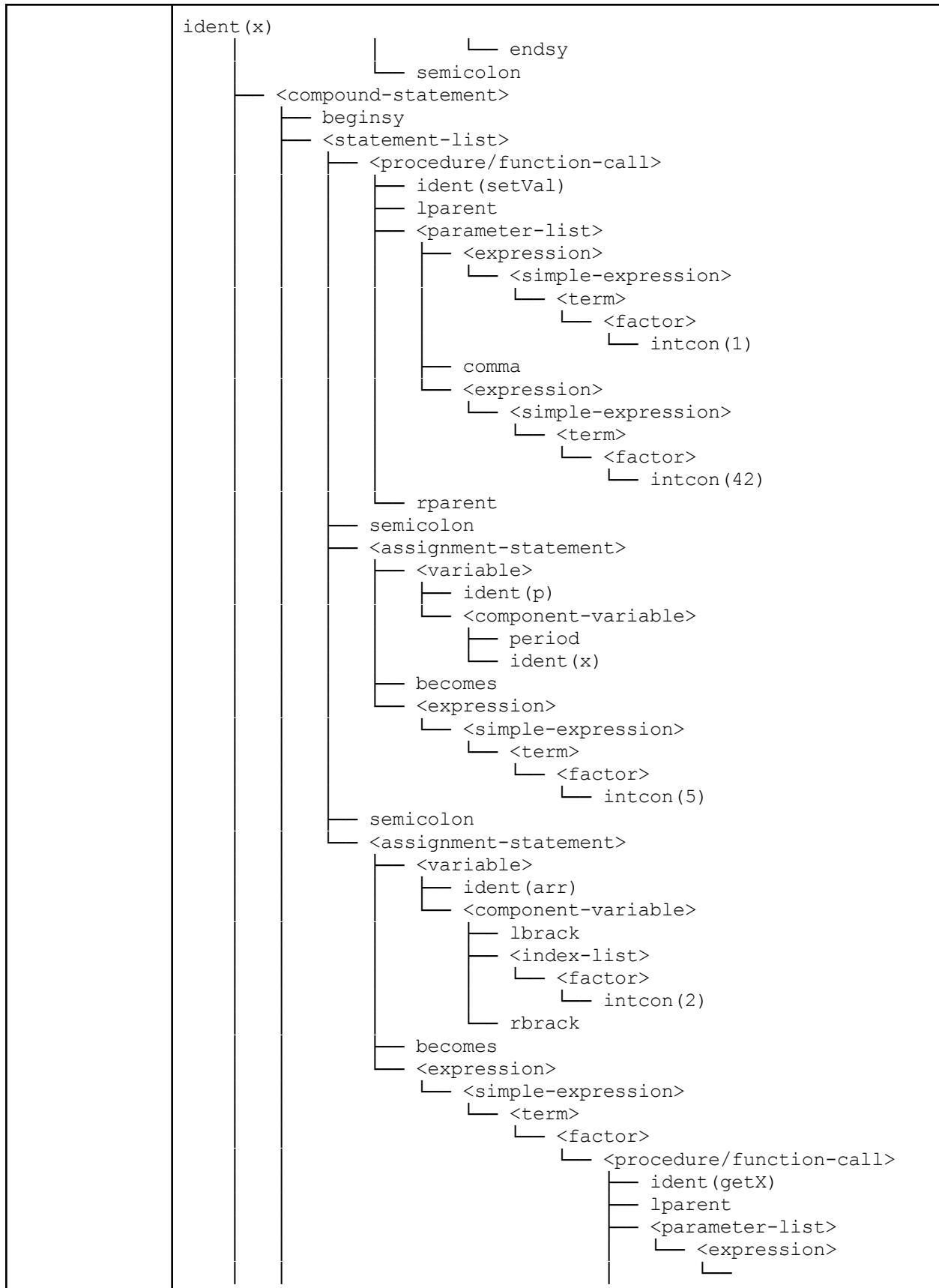


	<pre> └─ endsy └─ period </pre>
input-4.txt	<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident (ControlFlow) │ └── semicolon ├─ <declaration-part> │ └─ <var-declaration> │ ├── varsy │ ├── <identifier-list> │ │ └── ident (i) │ ├── colon │ ├── <type> │ │ └── ident (integer) │ └── semicolon ├─ <compound-statement> │ ├── beginsy │ └─ <statement-list> │ ├── <assignment-statement> │ │ ├── ident (i) │ │ ├── becomes │ │ └─ <expression> │ │ └─ <simple-expression> │ │ └─ <term> │ │ └─ <factor> │ │ └─ intcon (0) │ ├── semicolon │ └─ <while-statement> │ ├── whilesy │ └─ <expression> │ ├── <simple-expression> │ │ └─ <term> │ │ └─ <factor> │ │ └─ ident (i) │ ├── <relational-operator> │ │ └─ lss │ └─ <simple-expression> │ └─ <term> │ └─ <factor> │ └─ intcon (10) │ └─ dosy │ └─ <if-statement> │ ├── ifsy │ └─ <expression> │ ├── <simple-expression> │ │ └─ <term> │ │ ├── <factor> │ │ │ └─ ident (i) │ │ ├── <multiplicative-operator> │ │ │ └─ imod │ │ └─ <factor> │ │ └─ intcon (2) │ └─ <relational-operator> │ └─ eql </pre>

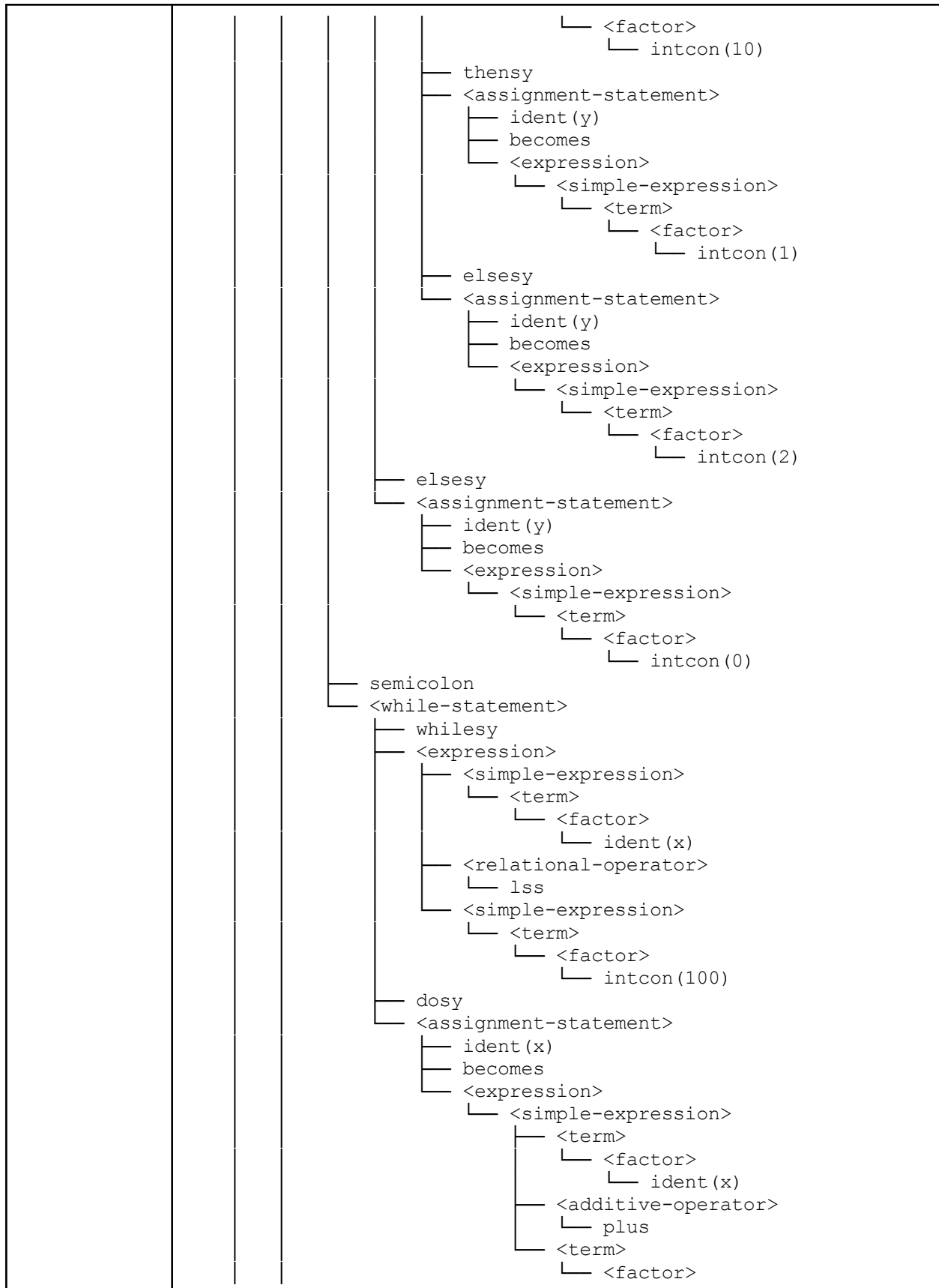
	<pre> └─ <simple-expression> └─ <term> └─ <factor> └─ intcon(0) thensy <procedure/function-call> ├─ ident(writeln) ├─ lparent ├─ <parameter-list> │ └─ <expression> │ └─ <simple-expression> │ └─ <term> │ └─ <factor> │ └─ ident(i) └─ rparent semicolon <assignment-statement> ├─ ident(i) ├─ becomes └─ <expression> └─ <simple-expression> └─ <term> └─ <factor> └─ ident(i) └─ <additive-operator> └─ plus └─ <term> └─ <factor> └─ intcon(1) semicolon endsy period </pre>
input-5.txt	<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(Complex) │ └─ semicolon └─ <declaration-part> ├── <type-declaration> │ ├── typesy │ ├── ident(Point) │ ├── eql │ └─ <type> │ └─ <record-type> │ ├── recordsy │ └─ <field-list> │ ├── <field-part> │ │ ├── <identifier-list> │ │ │ └─ ident(x) │ │ ├── colon │ │ └─ <type> │ │ └─ ident(integer) │ └─ semicolon └─ <field-part> └─ <identifier-list> </pre>



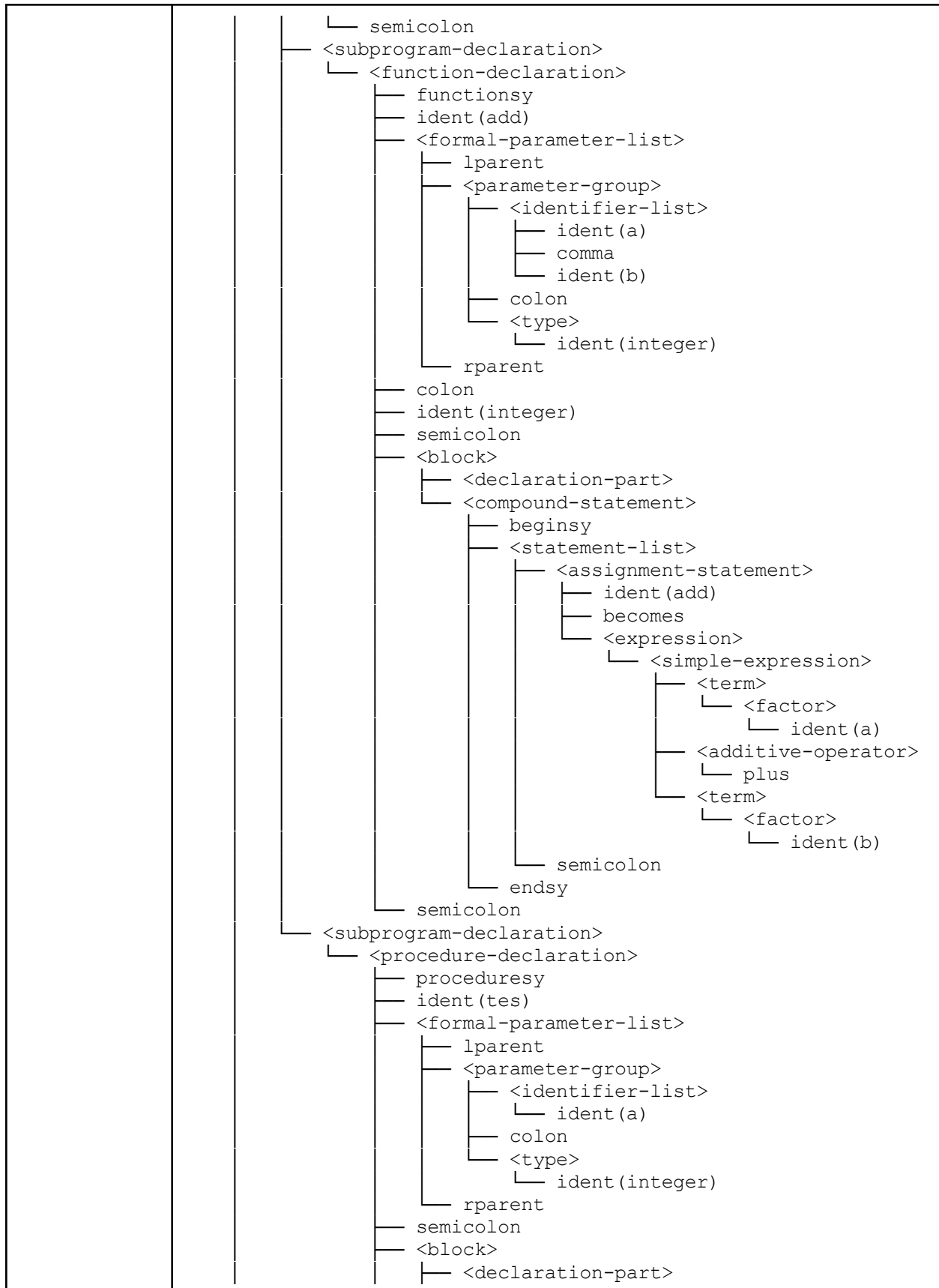


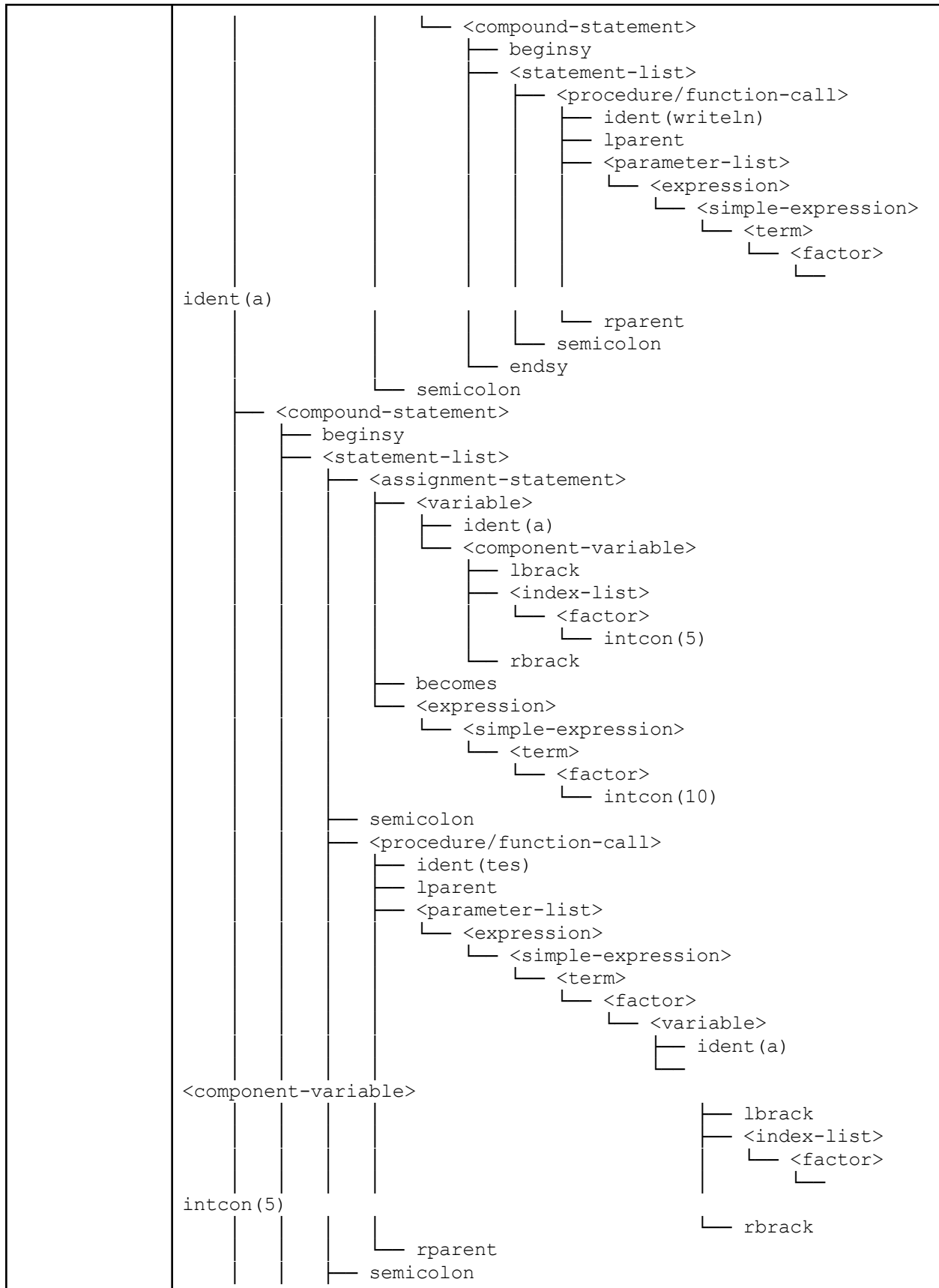


	<pre> <simple-expression> _ <factor> _ <term> _ ident (p) _ endsy _ period _ rparent </pre>
input-6.txt	<pre> <program> _ <program-header> _ programsy _ ident (Flow) _ semicolon _ <declaration-part> _ <var-declaration> _ varsy _ <identifier-list> _ ident (x) _ comma _ ident (y) _ colon _ <type> _ ident (integer) _ semicolon _ <compound-statement> _ beginsy _ <statement-list> _ <if-statement> _ ifsy _ <expression> _ <simple-expression> _ <term> _ <factor> _ ident (x) _ <relational-operator> _ gtr _ <simple-expression> _ <term> _ <factor> _ intcon (0) _ thensy _ <if-statement> _ ifsy _ <expression> _ <simple-expression> _ <term> _ <factor> _ ident (x) _ <relational-operator> _ gtr _ <simple-expression> _ <term> </pre>

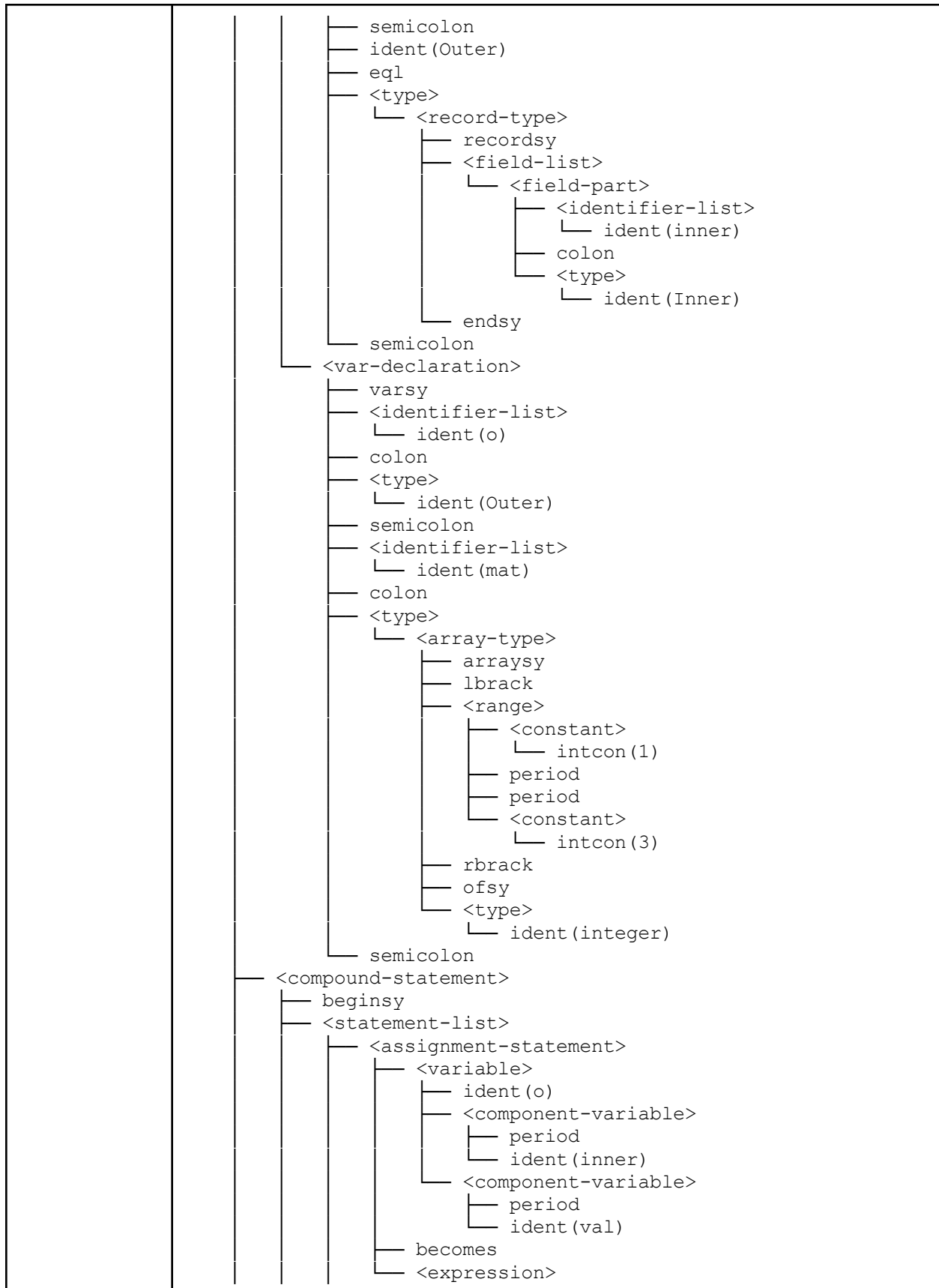


	<pre> └─ endsy └─ period └─ intcon(1) </pre>
input-7.txt	<pre> <program> └─ <program-header> └─ programsy └─ ident (SubProgramTest) └─ semicolon └─ <declaration-part> └─ <const-declaration> └─ constsy └─ ident (A) └─ eql └─ <constant> └─ intcon(100) └─ semicolon └─ ident (B) └─ eql └─ <constant> └─ string(yaya) └─ semicolon └─ ident (C) └─ eql └─ <constant> └─ plus └─ realcon(10.0) └─ semicolon └─ <var-declaration> └─ varsy └─ <identifier-list> └─ ident(res) └─ colon └─ <type> └─ ident(integer) └─ semicolon └─ <identifier-list> └─ ident(a) └─ colon └─ <type> └─ <array-type> └─ arraysy └─ lbrack └─ <range> └─ <constant> └─ intcon(1) └─ period └─ period └─ <constant> └─ intcon(100) └─ rbrack └─ ofsy └─ <type> └─ ident(integer) </pre>

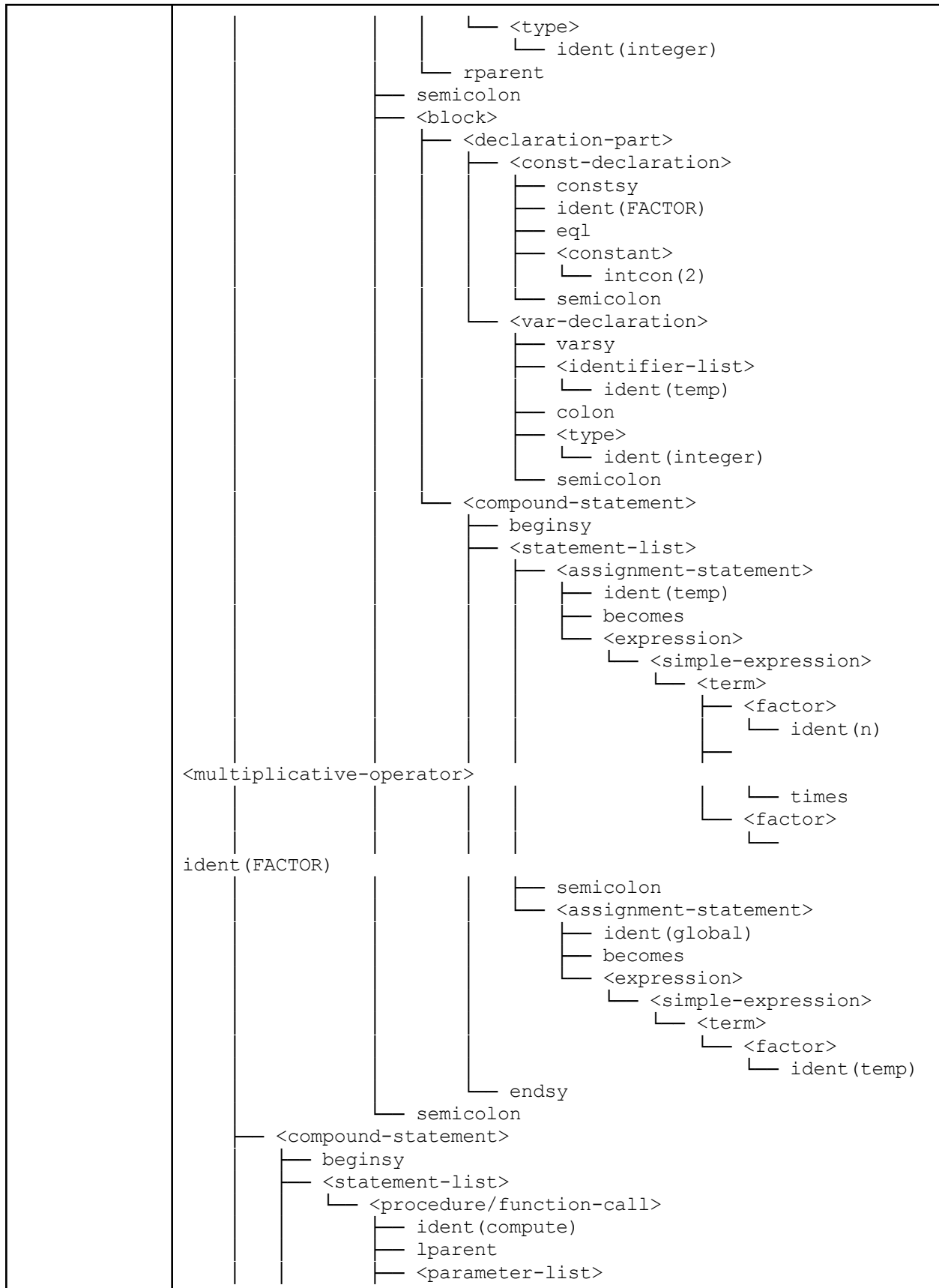




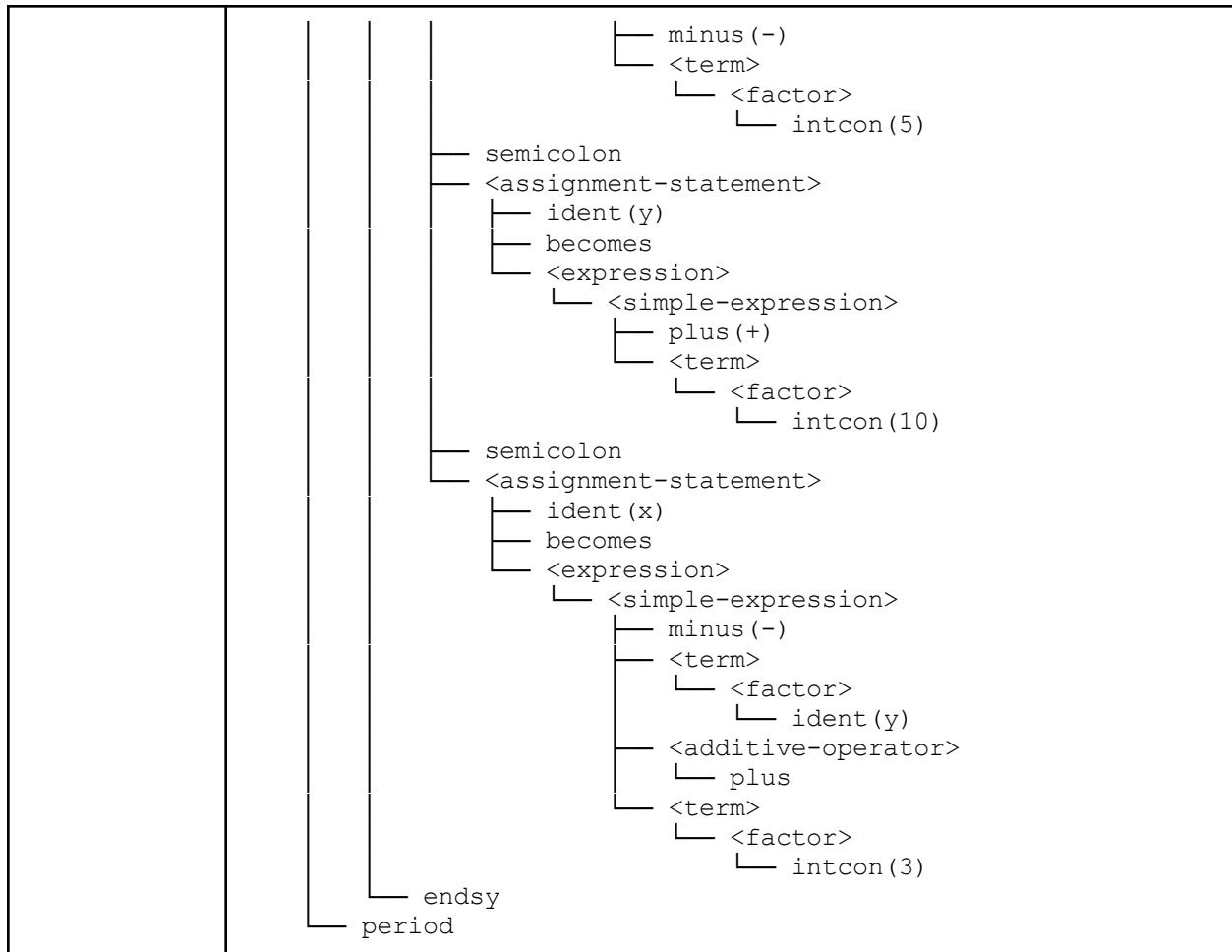
[illegible]



	<pre> └─ <simple-expression> └─ <term> └─ <factor> └─ intcon(7) └─ semicolon └─ <assignment-statement> └─ <variable> └─ ident(mat) └─ <component-variable> └─ lbrack └─ <index-list> └─ <factor> └─ intcon(1) └─ rbrack └─ becomes └─ <expression> └─ <simple-expression> └─ <term> └─ <factor> └─ <variable> └─ ident(o) └─ <component-variable> └─ period └─ ident(inner) └─ <component-variable> └─ period └─ ident(val) └─ endsy └─ period </pre>
input-9.txt	<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(Locals) │ └─ semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ └─ ident(global) │ │ ├── colon │ │ ├── <type> │ │ │ └─ ident(integer) │ │ └─ semicolon │ └─ <subprogram-declaration> │ └─ <procedure-declaration> │ ├── proceduresy │ ├── ident(compute) │ ├── <formal-parameter-list> │ │ ├── lparent │ │ ├── <parameter-group> │ │ │ ├── <identifier-list> │ │ │ │ └─ ident(n) │ │ └─ colon </pre>



	<pre> └─ <expression> └─ <simple-expression> └─ <term> └─ <factor> └─ intcon(5) └─ rparent └─ endsy └─ period </pre>
input-10.txt	<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(Edge) │ └── semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ ├── ident(x) │ │ │ ├── comma │ │ │ └── ident(y) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ └── semicolon │ └── <subprogram-declaration> │ └─ <procedure-declaration> │ ├── proceduresy │ ├── ident(greet) │ ├── semicolon │ └─ <block> │ ├── <declaration-part> │ │ └─ <compound-statement> │ │ ├── beginsy │ │ └─ <statement-list> │ │ └─ <assignment-statement> │ │ ├── ident(x) │ │ ├── becomes │ │ └─ <expression> │ │ └─ <simple-expression> │ │ └─ <term> │ │ └─ <factor> │ │ └─ intcon(1) │ └─ endsy │ └─ semicolon └─ <compound-statement> ├── beginsy └─ <statement-list> ├── <procedure/function-call> │ └─ ident(greet) ├── semicolon └─ <assignment-statement> ├── ident(x) ├── becomes └─ <expression> └─ <simple-expression> </pre>



BAB 4

Kesimpulan dan Saran

4.1 Kesimpulan

Pada Milestone 2 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata ini, kami telah berhasil mengimplementasikan syntax analyzer (parser) untuk bahasa pemrograman Arion menggunakan pendekatan Recursive Descent. Implementasi ini mencakup proses pembentukan Parse Tree dari deretan token yang dihasilkan oleh lexer pada milestone sebelumnya, dengan grammar bahasa Arion yang di-hardcode secara terstruktur dalam program. Parser yang dibangun mampu mengenali berbagai konstruksi sintaksis seperti deklarasi konstanta, tipe, variabel, prosedur, dan fungsi; statement kondisional (if-then-else, case-of); perulangan (while-do, repeat-until, for-to/downto); ekspresi dengan operator precedence yang sesuai; serta akses komponen array dan field record sesuai spesifikasi yang digunakan. Pada parser, kami juga telah melengkapi mekanisme error handling yang dapat untuk memberikan pesan ketika ditemukan kesalahan yang bersifat sintaksis pada input.

4.2 Saran

Kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

BillyO	rajin - rajin cek qna
Nathan	Asdnaskdjaskjdlkasjdklasjdkaskdjaksjas :DDDDD
Azri	Kapan ya sejago Sandwicheese
Rasyad	Kak william tolong ini nya di anuin

Lampiran

Repository: <https://github.com/AzriVz/CTF-Tubes-IF2224-2026>

Diagram Transisi DFA:

<https://drive.google.com/file/d/1Q8MlnukMm2jU2dEt5GNVJjzSQ2MCpIng/view?usp=sharing>

Tabel Pembagian Tugas:

Nama	NIM	Workload	Persentase
Billy Ontoseno Irawan	13524121	Laporan (Bab 1 dan 2)	25%
Nathaniel Christian	13524122	Parser.cpp Parser.hpp main.cpp	35%
Azri Arzaq Pohan	13523139	Laporan (Bab 3 dan 4) Test cases	20%
Rasyad Satyatma	13524142	Test cases	20%

Referensi

[CS 241 Lecture 9 - Maximal Munch and Context Free Grammars With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)

[CS 241 Lecture 11 - Top Down Parsing, First and Follow With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot](#)